

Efficient Data Storage Systems

University of Minho

Guide 5

Storage space reduction: Deduplication

2026

The main goal of this guide is to design and implement a file system with data deduplication capabilities.

1 Setup

Use the VM and FUSE environment for the exercises next. Adapt the code provided along with this guide or adapt your own code (based on *Guide0* and *Guide2*). Also, install LibSSL with:

```
sudo apt install libssl-dev
```

2 Exercises

1. Start by creating an in-memory index that maps the hash sum of a given file to its filename. Consider the following requirements.
 - Assume a flat file system (no subdirectories) and a workload where files are neither updated nor deleted after being closed.
 - Use OpenSSL's SHA-512 function for calculating the hash sum of files' content. An example for calculating a hash sum is included in the codebase provided with this guide (files *passthrough.c* and *passthrough_helpers.h*).
 - The index should be updated after each file is closed. For files with identical content, only the first one is added to the index. Only close requests for files opened for writing should be included. You can check for files opened for writing with:

```
if((fi->flags & O_WRONLY) || (fi->flags & O_RDWR)){ ... }
```

Try out your program with small files created by you and use *printf()* or a log file to debug your program and check the operations done to the index.

2. Keeping the assumptions made for exercise 1, extend your file system to avoid storing duplicate files (*i.e.*, with the same content, even if these have different filenames). Note that you will require an additional metadata structure mapping logical file names to the physical name of stored files.
 - Do not remove duplicate files with `unlink`, instead truncate the files to zero bytes. This way the files will still be searchable at their directory.

```
int truncate(const char *path, off_t length);
```

 - Note that FUSE only issues reads for a file, after opening it, if the file has more than zero bytes. The size of a file is checked with FUSE's *getattr* function. After removing (*i.e.*, truncating to zero bytes) duplicates, both FUSE's *open* and *getattr* functions need to map logical file attributes (*e.g.*, path, size) into the corresponding physical ones.

3 Extras

1. Add support for file deletion by adding reference counting to the index structure.
2. Change your implementation to use fixed-size chunking instead of whole-file chunking as the unit of deduplication.

Hint: Follow an inline deduplication design and start with a block size of 4 KB.

3. Add support for workloads that update files in place.

Learning outcomes

Implement deduplication at the file system level. Understand and implement metadata structures for supporting data deduplication. Learn performance, resource usage, and space savings implications of using deduplication.